

WOMEN'S TENNIS

ANALYTICS

By: Steven Carvalho,
Prince Chinyama,
Kamalesh Kumar,
Ryan O'Malley

Introduction	03
• Background	
• Management	
Key	
Components	03
• Data Dictionary	
• Entity Relationship Diagram	
• Data in Database	
Main Question	03
• SQL Code	
• Results	
Sub-Question 1	03
• SQL Code	
• Results	
Sub-Question 2	03
• SQL Code	
• Results	
Sub-Question 3	03
• SQL Code	
• Results	
Conclusion and Reflection	03



Agenda

BACKGROUND

- Most sports following the path of “Moneyball”
- Tennis has been one of the less analytically-driven sports so far
- Project aims to determine the effectiveness of analytics in tennis

PROJECT MANAGEMENT

1. Emphasis on completing tasks in a timely manner, and in order of what topic was covered in class
2. Stayed in constant communication as a group to ensure there was a common understanding
3. Occasional check-ins with the professor to confirm any intuitions or to seek guidance

DATA DICTIONARY - MATCHES

Column Name	Description	Data Type	Units / Format/Examples	winner_seed	Seed number of the winning player	Integer	1, 7
tourney_id	Unique identifier for the tournament	String(Primary Key)	YYYY-NNNN "2023-560"	winner_entry	Entry type of winner (e.g., WC = Wild Card, Q = Qualifier)	String	"WC", "Q", "LL"
tourney_name	Name of the tournament	String	US Open/Wimbledon/Roland Garros	winner_name	Full name of the winning player	String	"Serena Williams"
surface	Court surface type (e.g. Hard, Clay, Grass)	String	Clay, Grass, Hard	winner_hand	Playing hand of the winner (R = Right, L = Left, U = Unknown)	String	"R", "L"
draw_size	Number of players in the draw	Integer	128, 64	winner_ht	Height of the winner	Integer	cm
tourney_level	Tournament level (e.g., G = Grand Slam, M = Masters, I =	String	"G" = Grand Slam, "M" = Masters	winner_ioc	IOC country code of the winner	String	3 letters; country abbreviation "USA", "FRA"
tourney_date	Start date of the tournament	Integer	YYYYMMDD "20250826"	winner_age	Age of the winner	Float	Years
match_num	Match number within the tournament	Integer (Primary Key)	312	loser_id	Unique identifier for the losing player	Integer Foreign Key)	20567
winner_id	Unique identifier for the winning player	Integer (Foreign Key)	10345	loser_seed	Seed number of the losing player	Integer	14

DATA DICTIONARY - MATCHES

loser_entry	Entry type of loser	String	"Q", "WC"
loser_name	Full name of the losing player	String	"Naomi Osaka"
loser_hand	Playing hand of the loser	String	"R"
loser_ht	Height of the loser	Integer	cm
loser_ioc	IOC country code of the loser	String	3 letters; country abbreviation "USA"
loser_age	Age of the loser	Float	Years
score	Final score of the match	String	"6-4 3-6 7-6"
best_of	Best of sets (3 for WTA)	Integer	3
round	Round of the match (e.g., R32, QF, SF, F)	String	"R32", "QF", "SF", "F"
minutes	Match duration	Integer	Minutes
w_ace	Aces by the winner	Integer	
w_df	Double faults by the winner	Integer	
w_svpt	Total serve points played by winner	Integer	
w_1stIn	First serves in by the winner	Integer	
w_1stWon	First serve points won by the winner	Integer	
w_2ndWon	Second serve points won by the winner	Integer	

w_SvGms	Service games played by the winner	Integer	
w_bpSaved	Break points saved by the winner	Integer	
w_bpFaced	Break points faced by the winner	Integer	
l_ace	Aces by the loser	Integer	
l_df	Double faults by the loser	Integer	
l_svpt	Total serve points played by loser	Integer	
l_1stIn	First serves in by the loser	Integer	
l_1stWon	First serve points won by the loser	Integer	
l_2ndWon	Second serve points won by the loser	Integer	
l_SvGms	Service games played by the loser	Integer	
l_bpSaved	Break points saved by the loser	Integer	
l_bpFaced	Break points faced by the loser	Integer	
winner_rank	WTA rank of the winner at match time	Integer	
winner_rank_points	Ranking points of the winner	Integer	
loser_rank	WTA rank of the loser at match time	Integer	
loser_rank_points	Ranking points of the loser	Integer	

DATA DICTIONARY - PLAYERS

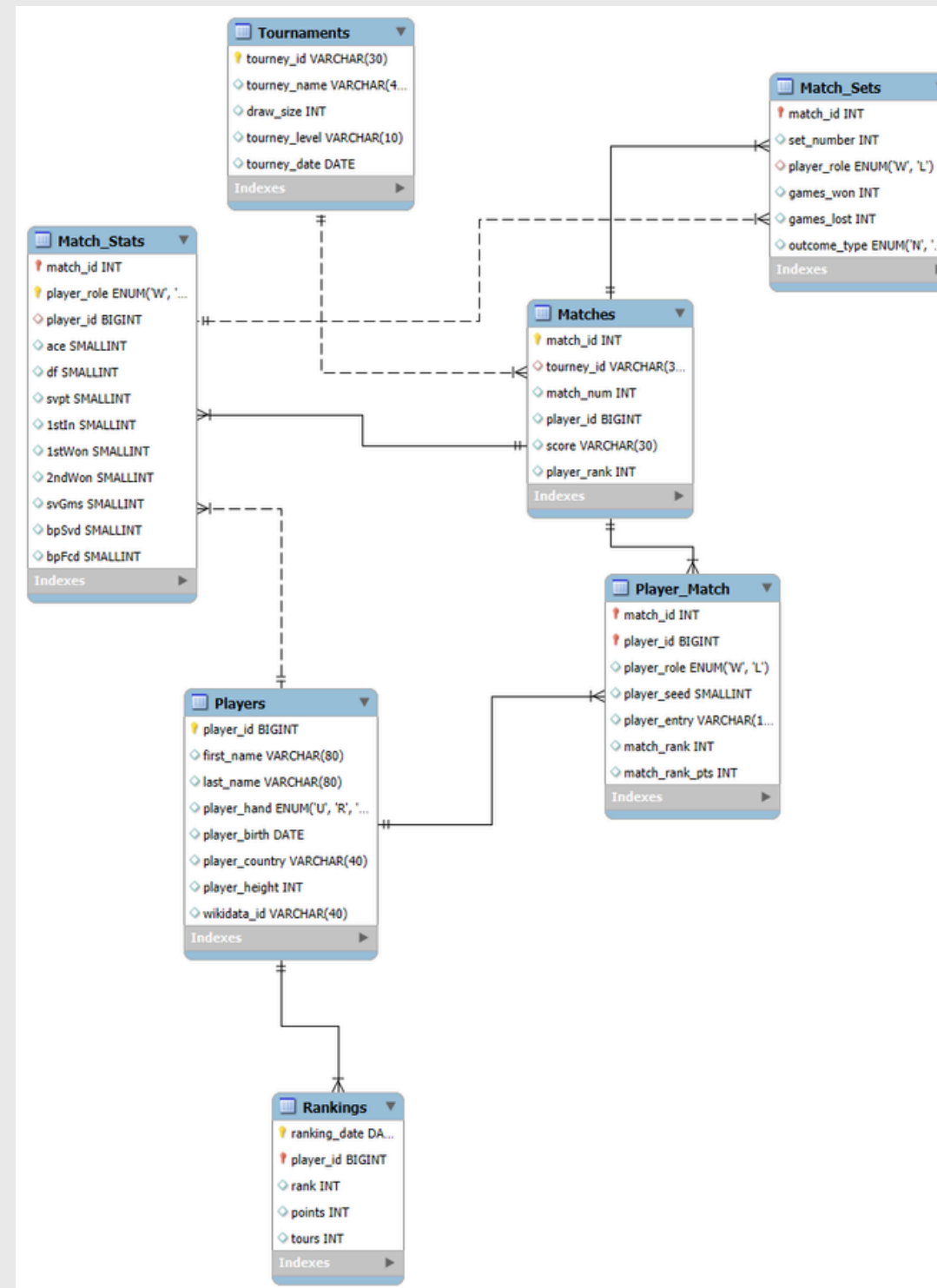
Column Name	Description	Data Type	Units / Format
player_id	Primary key for joins with matches and rankings	Integer (Primary Key)	
name_first	Player's first name	String	"Naomi"
name_last	Player's last name/surname	String	"Osaka"
hand	Players' playing hand	String	R/L/U (Right/Left/Unknown)
dob	Date of birth	String	YYYYMMDD
ioc	3-Letter Country Code	String	ABC
height	Player height in centimeters	Integer	cm
wikidata_id	Present in some repos/eras for enrichment	String	

DATA DICTIONARY - RANKINGS

Column Name	Description	Data Type	Units / Format
ranking_date	Effective date of the weekly rankings list.	String (Primary Key)	YYYYMMDD
rank	Position in the ranking.	Integer	
player_id	ID matching wta_players	String (Primar Key, Foreign Key)	
rank	Player's position in ranking (1 is the top ranking)	Integer	
points	Ranking points at the date.	Integer	

ENTITY RELATIONSHIP DIAGRAM

- Included leveraging AI to parse the “score” attribute since the column was a multi-valued attribute



- “match_id” attribute created to simplify initial composite key of “tourney_id” and “match_num”

MAIN QUESTION

What is the prime age (or age range) of a tennis player?

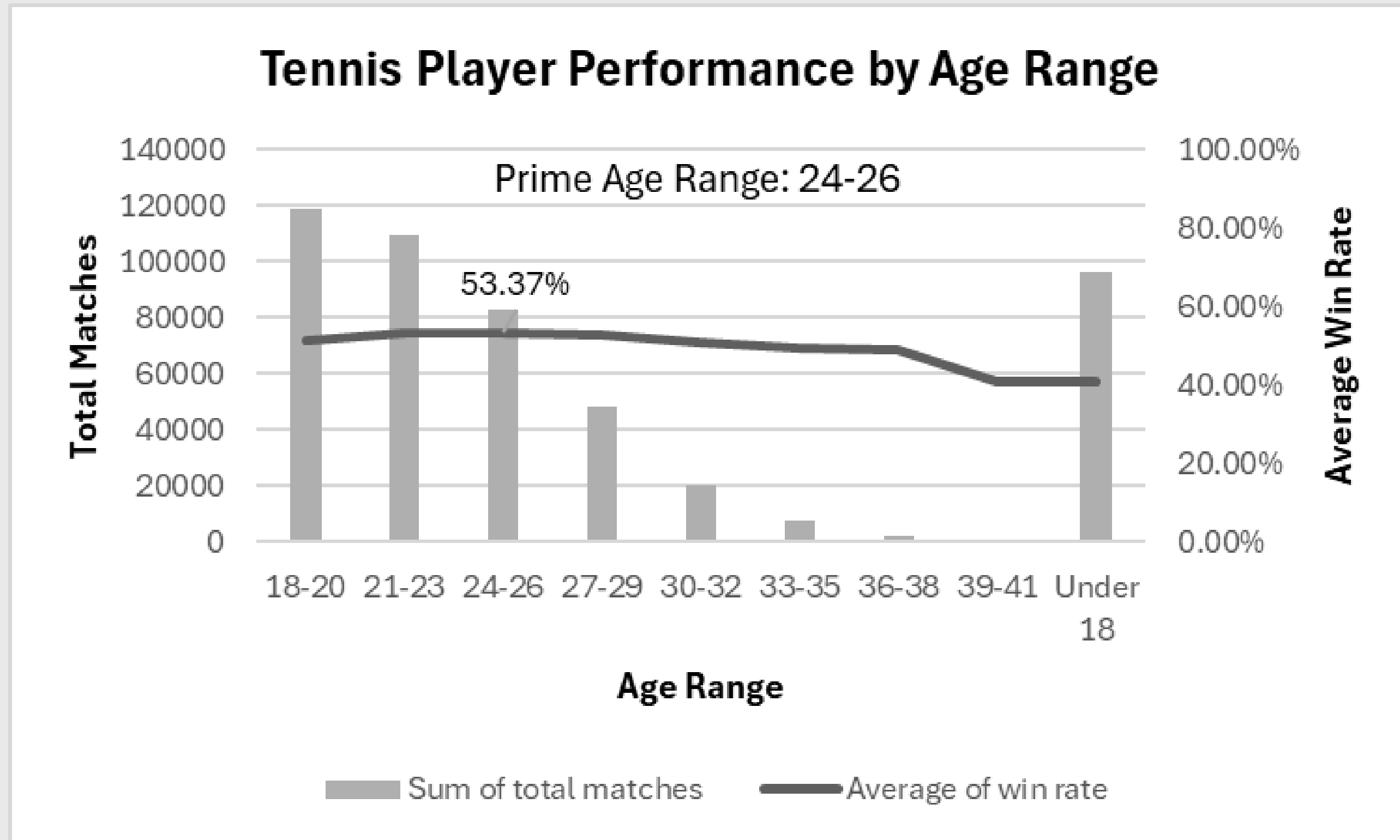
Based on this question, sub-questions revolving around the age of a tennis player were also developed and answered.



SQL CODE

```
FOR PRIME AGE RANGE:
WITH all_matches AS (
  SELECT winner_id AS player_id, winner_age AS age, 1 AS is_win
  FROM stg_matches_raw
  WHERE winner_age IS NOT NULL
UNION ALL
  SELECT loser_id AS player_id, loser_age AS age, 0 AS is_win
  FROM stg_matches_raw
  WHERE loser_age IS NOT NULL), binned AS (
  SELECT
    CASE
      WHEN age < 18 THEN 'Under 18'
      ELSE CONCAT(FLOOR(age/3)*3, '-', FLOOR(age/3)*3 + 2)
    END AS age_range,
    is_win
  FROM all_matches)
SELECT age_range, ROUND(SUM(is_win) * 1.0 / COUNT(*), 3) AS win_rate, COUNT(*) AS
total_matches FROM binned GROUP BY age_range
HAVING total_matches >= 30
ORDER BY win_rate DESC;
```

RESULTS



COMPARATIVE AGE STATISTICS

- Young Players: 20 or younger
- Early Prime: 21-23
- Prime: 24-26
- Late Prime: 27-32
- Veteran: 33 and older

Statistics:

- Players in group
- Matches played
- Average Aces
- Average Double Faults
- Average Service Points
- Percentage First Serve in
- Percentage First Serve Points Won
- Percentage Second Serve Points Won
- Percentage Break Points Saved
- Average Service Games
- Percentage Comeback Wins



SQL CODE

```
FOR COMPARATIVE AGE STATISTICS:
SELECT
    sub.age_group,
    COUNT(DISTINCT sub.player_id) AS players_in_group,
    COUNT(*) AS matches_played,

    AVG(sub.ace) AS avg_aces,
    AVG(sub.df) AS avg_double_faults,
    AVG(sub.svpt) AS avg_service_points,

    ROUND(AVG(CASE WHEN sub.svpt > 0 THEN sub.first_in / NULLIF(sub.svpt,0) ELSE NULL END) * 100, 2) AS pct_first_serve_in,
    ROUND(AVG(CASE WHEN sub.first_in > 0 THEN sub.first_won / NULLIF(sub.first_in,0) ELSE NULL END) * 100, 2) AS pct_first_serve_points_won,
    ROUND(AVG(CASE WHEN (sub.svpt - sub.first_in) > 0 THEN sub.second_won / NULLIF(sub.svpt - sub.first_in,0) ELSE NULL END) * 100, 2) AS pct_second_serve_points_won,
    ROUND(AVG(CASE WHEN sub.bp_faced > 0 THEN sub.bp_saved / NULLIF(sub.bp_faced,0) ELSE NULL END) * 100, 2) AS pct_break_points_saved,

    AVG(sub.sv_gms) AS avg_service_games,

    -- comeback win rate (won match after losing first set)
    ROUND(AVG(sub.comeback_win) * 100, 2) AS pct_comeback_wins

FROM (
    SELECT
        ms.*,
        p.dob,
        t.start_date,
        TIMESTAMPDIFF(YEAR, p.dob, t.start_date) AS age,

        -- 🔄 Updated age-group logic
        CASE
            WHEN TIMESTAMPDIFF(YEAR, p.dob, t.start_date) <= 20 THEN 'Young'
            WHEN TIMESTAMPDIFF(YEAR, p.dob, t.start_date) BETWEEN 21 AND 23 THEN 'Early-Prime'
            WHEN TIMESTAMPDIFF(YEAR, p.dob, t.start_date) BETWEEN 24 AND 26 THEN 'Prime'
            WHEN TIMESTAMPDIFF(YEAR, p.dob, t.start_date) BETWEEN 27 AND 32 THEN 'Late-Prime'
            ELSE 'Veteran'
        END AS age_group,

        -- comeback flag: player lost first set but won the match
        CASE
            WHEN EXISTS (
                SELECT 1
                FROM match_set_scores mss
                WHERE mss.tourney_id = ms.tourney_id
                AND mss.match_num = ms.match_num
                AND mss.player_id = ms.player_id
                AND mss.set_no = 1
                AND mss.set_result = 'L'
            )
            AND EXISTS (
                SELECT 1
                FROM match_set_scores mss2
                WHERE mss2.tourney_id = ms.tourney_id
                AND mss2.match_num = ms.match_num
                AND mss2.player_id = ms.player_id
                AND mss2.set_result = 'W'
            )
            THEN 1 ELSE 0
        END AS comeback_win

    FROM match_stats ms
    JOIN players p ON ms.player_id = p.player_id
    JOIN tournaments t ON ms.tourney_id = t.tourney_id
    WHERE p.dob IS NOT NULL
    AND t.start_date IS NOT NULL
    ) AS sub

GROUP BY sub.age_group
ORDER BY FIELD(sub.age_group, 'Young', 'Early-Prime', 'Prime', 'Late-Prime', 'Veteran');
```

RESULTS

age_group ▾	players_in_group ▾	matches_played ▾	avg_aces ▾	avg_double_faults ▾	avg_service_points ▾	pct_first_serve_in ▾	pct_first_serve_points_won ▾	pct_second_serve_points_won ▾	pct_break_points_saved ▾	avg_service_games ▾	pct_comeback_wins ▾
Young	4514	141600	0.1462	0.2431	4.2349	61.24	62.75	44.94	51.55	0.6199	1.74
Early-Prime	2203	92515	0.3744	0.5713	10.4845	61.89	63.05	45.47	51.88	1.5392	2.65
Prime	1396	70062	0.5738	0.8029	15.884	62.43	63.17	45.34	52.3	2.3422	2.86
Late-Prime	903	58830	0.8013	1.1679	23.3856	63.02	62.79	45.3	51.99	3.4573	2.87
Veteran	225	8661	0.8769	1.3423	28.0197	63.91	62.24	44.79	51.28	4.0933	4.08

YOUNGER PLAYERS

- Most Players in the Group
- Most Matches Played
- 2nd Lowest
Pct_First_Serve_Points_Won,
Pct_Second_Serve_Points_Won,
and Pct_Break_Points_Saved
- Lowest in all other categories

EARLY PRIME

- Highest Pct_Second_Serve_Points_Won

PRIME

- Highest_Pct_First_Serve_Points_Won
- Highest_Pct_Break_Points_Saved

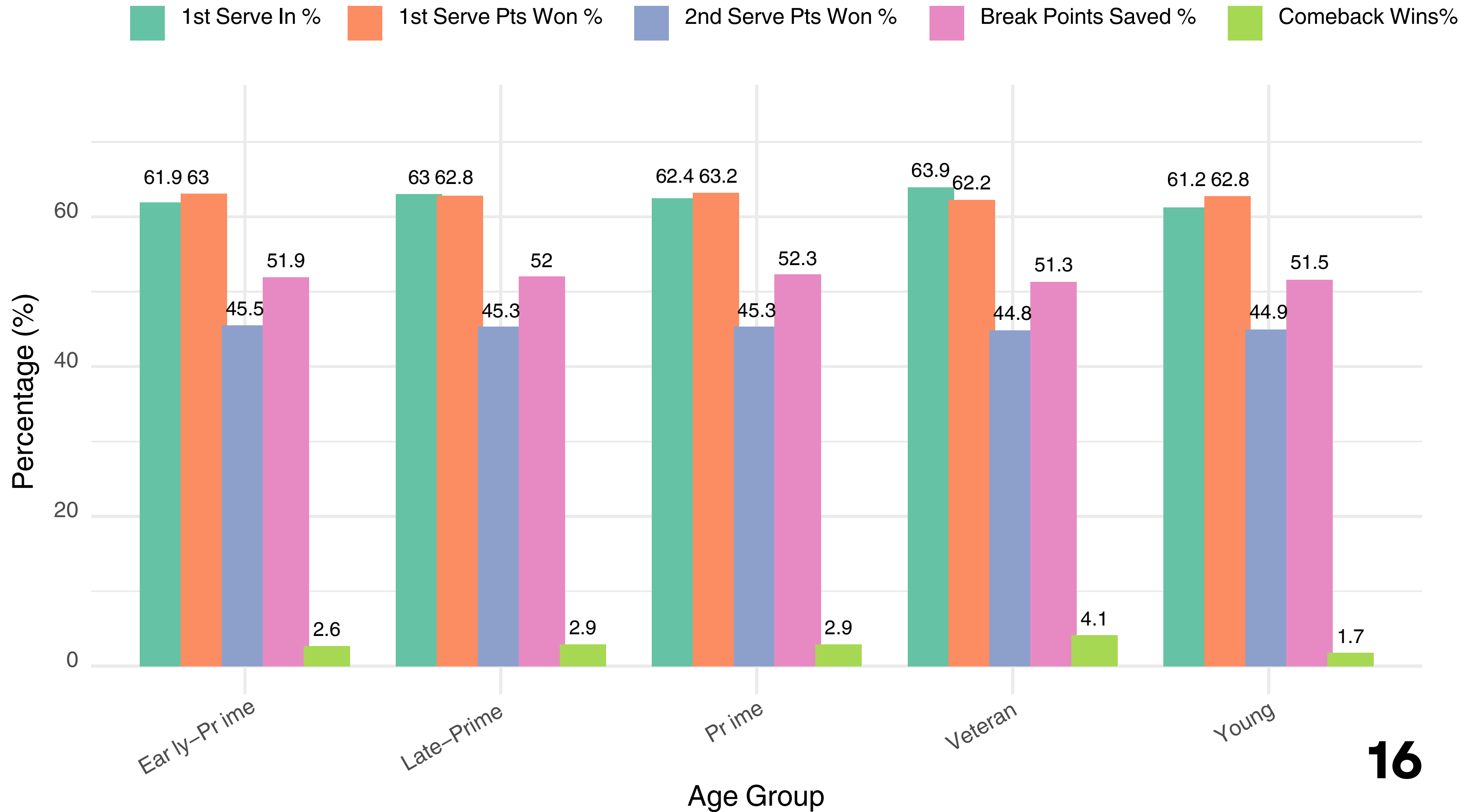
LATE PRIME

- Does not lead or have the lowest in
any category, very close in
pct_second_serve_points_won to
prime

VETERAN

- Highest in avg_service_games,
pct_comeback_wins, avg_service_points,
pct_first_serve_in, avg_double_faults, and
average aces

Serve & Comeback Performance by Age Group



ADVANTAGES OF YOUNGER PLAYERS HYPOTHESES

- **Rapid Skill Development/Ranking Progression**
- **Surprise Factor/Upset Potential**

SQL CODE

FOR RANKING PROGRESSION BETWEEN TOP 5 YOUNG PLAYERS:

```
SELECT player_name, MIN(ranking) AS highest_ranking_achieved,  
       MIN(age_at_best_rank) AS age_at_best_ranking
```

```
FROM (SELECT
```

```
    player_name,  
    ranking,  
    age AS age_at_best_rank
```

```
FROM (SELECT winner_name AS player_name,  
            winner_rank AS ranking,  
            winner_age AS age,
```

```
        ROW_NUMBER() OVER (PARTITION BY winner_name ORDER BY winner_rank ASC) as rank_sequence
```

```
FROM stg_matches_raw
```

```
WHERE winner_age <= 20 AND winner_age > 0 AND winner_age IS NOT NULL
```

```
AND winner_rank > 0 AND winner_rank IS NOT NULL
```

```
UNION ALL
```

```
SELECT loser_name AS player_name, loser_rank AS ranking, loser_age AS age,
```

```
        ROW_NUMBER() OVER (PARTITION BY loser_name ORDER BY loser_rank ASC) as rank_sequence
```

```
FROM stg_matches_raw
```

```
WHERE loser_age <= 20 AND loser_age > 0 AND loser_age IS NOT NULL
```

```
AND loser_rank > 0 AND loser_rank IS NOT NULL
```

```
) all_rankings
```

```
WHERE rank_sequence = 1
```

```
) best_rankings
```

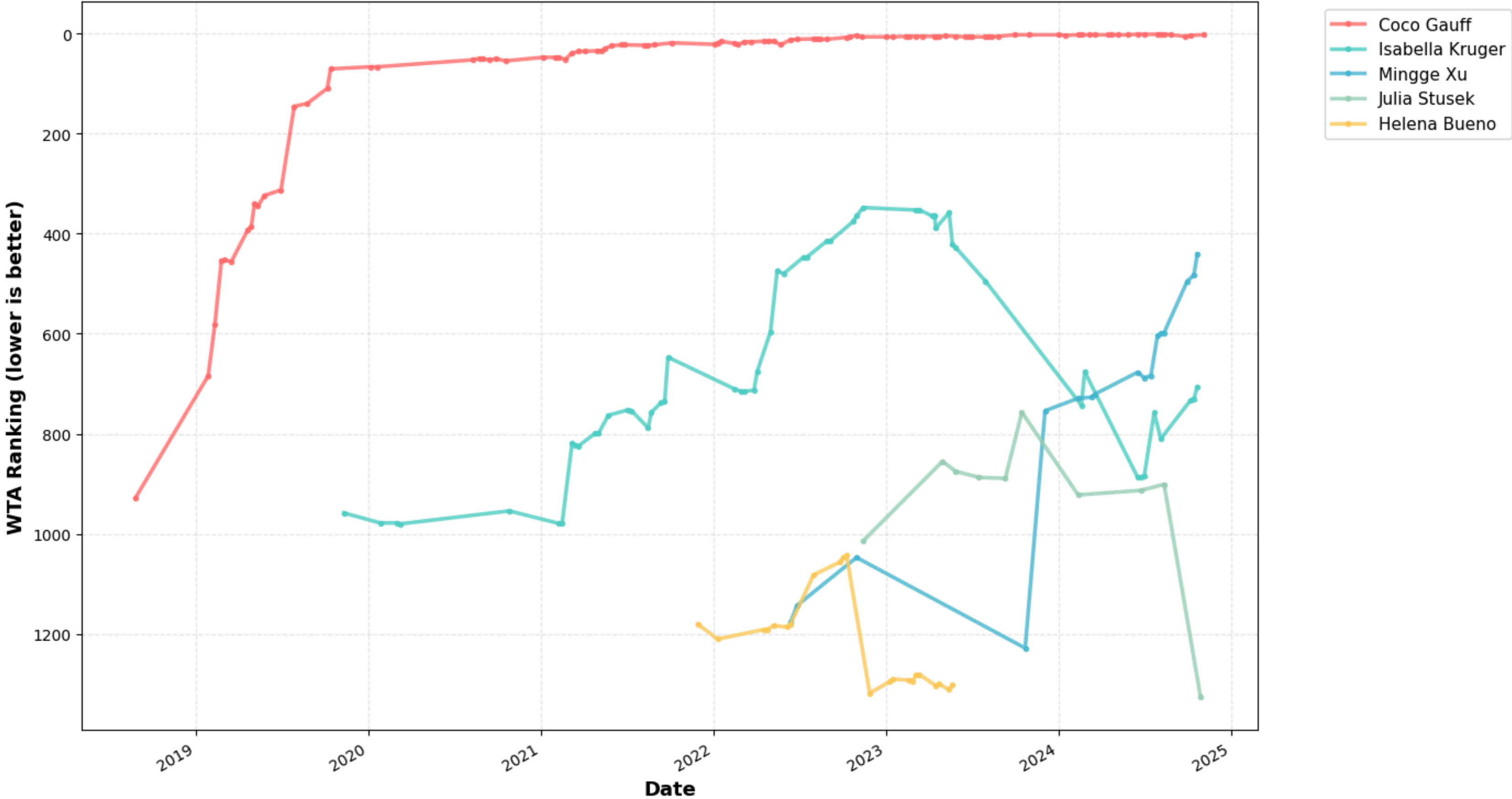
```
GROUP BY player_name
```

```
HAVING MIN(ranking) IS NOT NULL
```

```
ORDER BY highest_ranking_achieved ASC
```

```
LIMIT 5;
```

Top 5 Young WTA Players — Ranking Progression Comparison



SQL CODE

```
FOR UPSET RATE BY AGE GROUP:
    SELECT
        CASE
            WHEN TIMESTAMPDIFF(YEAR, p.dob, t.start_date) <= 20 THEN 'Young (≤20)'
            WHEN TIMESTAMPDIFF(YEAR, p.dob, t.start_date) BETWEEN 21 AND 25 THEN '21-25'
            WHEN TIMESTAMPDIFF(YEAR, p.dob, t.start_date) BETWEEN 26 AND 30 THEN '26-30'
            ELSE 'Veteran (31+)'
        END AS age_group,

        COUNT(*) AS matches_as_underdog,

        -- Upset wins (beating higher-ranked opponent)
        SUM(CASE WHEN pm.role = 'W' THEN 1 ELSE 0 END) AS upset_wins,

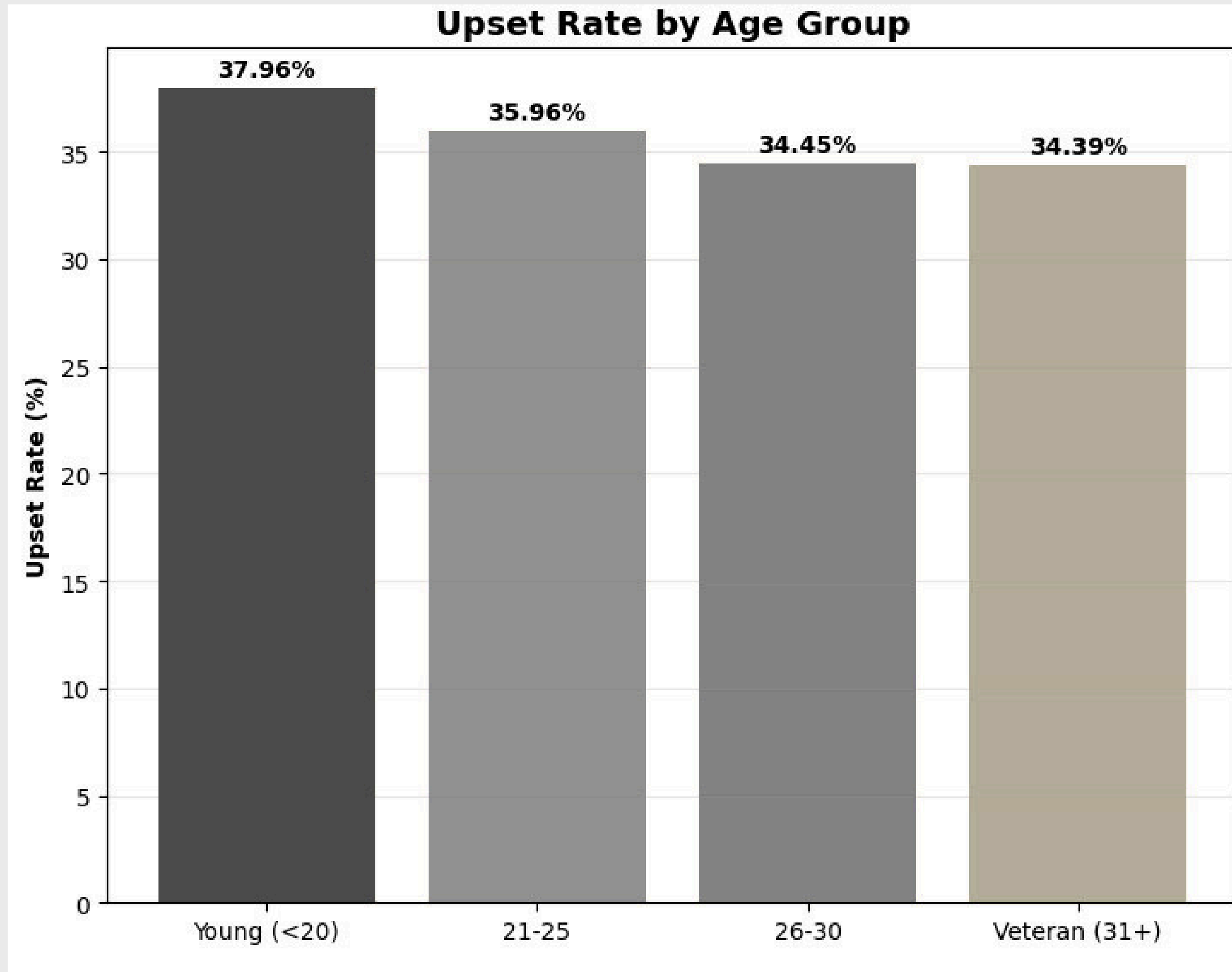
        -- Upset rate
        ROUND(SUM(CASE WHEN pm.role = 'W' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS upset_rate,

        -- Average ranking gap in upsets
        ROUND(AVG(CASE WHEN pm.role = 'W' THEN opponent_rank - pm.pre_rank ELSE NULL END), 1) AS avg_ranking_gap_in_upsets,

        -- Biggest upset (largest ranking gap overcome)
        MAX(CASE WHEN pm.role = 'W' THEN opponent_rank - pm.pre_rank ELSE 0 END) AS biggest_upset_gap

    FROM player_match pm
    JOIN players p ON pm.player_id = p.player_id
    JOIN tournaments t ON pm.tourney_id = t.tourney_id
    JOIN (
        -- Get opponent's ranking for each match
        SELECT
            tourney_id,
            match_num,
            player_id,
            pre_rank AS opponent_rank
        FROM player_match
    ) AS opponents ON pm.tourney_id = opponents.tourney_id
    AND pm.match_num = opponents.match_num
    AND pm.player_id != opponents.player_id
    WHERE p.dob IS NOT NULL
    AND t.start_date IS NOT NULL
    AND pm.pre_rank IS NOT NULL
    AND opponents.opponent_rank IS NOT NULL
    AND pm.pre_rank > opponents.opponent_rank -- Player is lower ranked (underdog)

    GROUP BY age_group
    ORDER BY FIELD(age_group, 'Young (≤20)', '21-25', '26-30', 'Veteran (31+)');
```

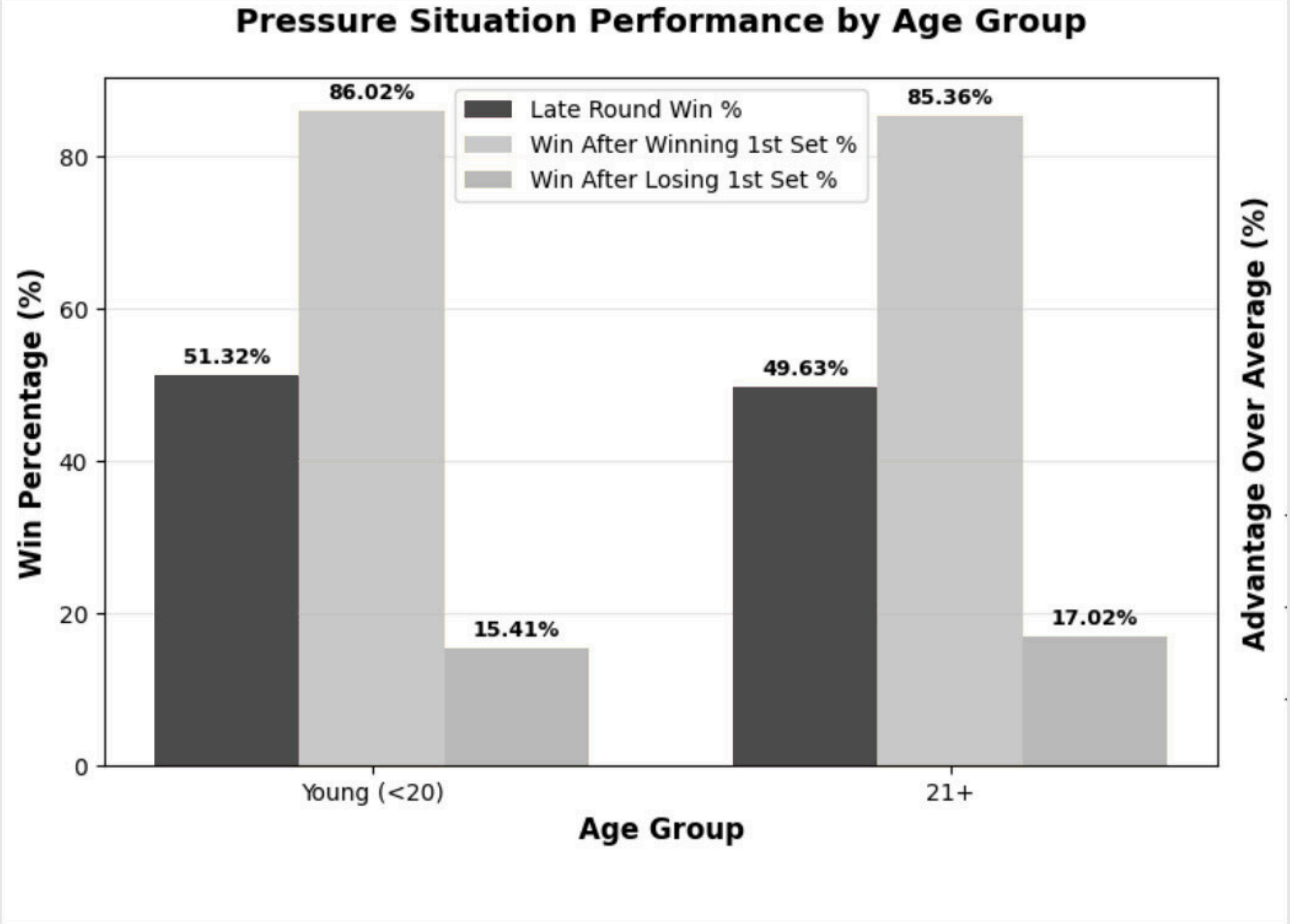


DISADVANTAGES OF YOUNGER PLAYERS HYPOTHESES

- **Experience Deficit in Important Matches**
- **Less ability to handle pressure**

SQL CODE

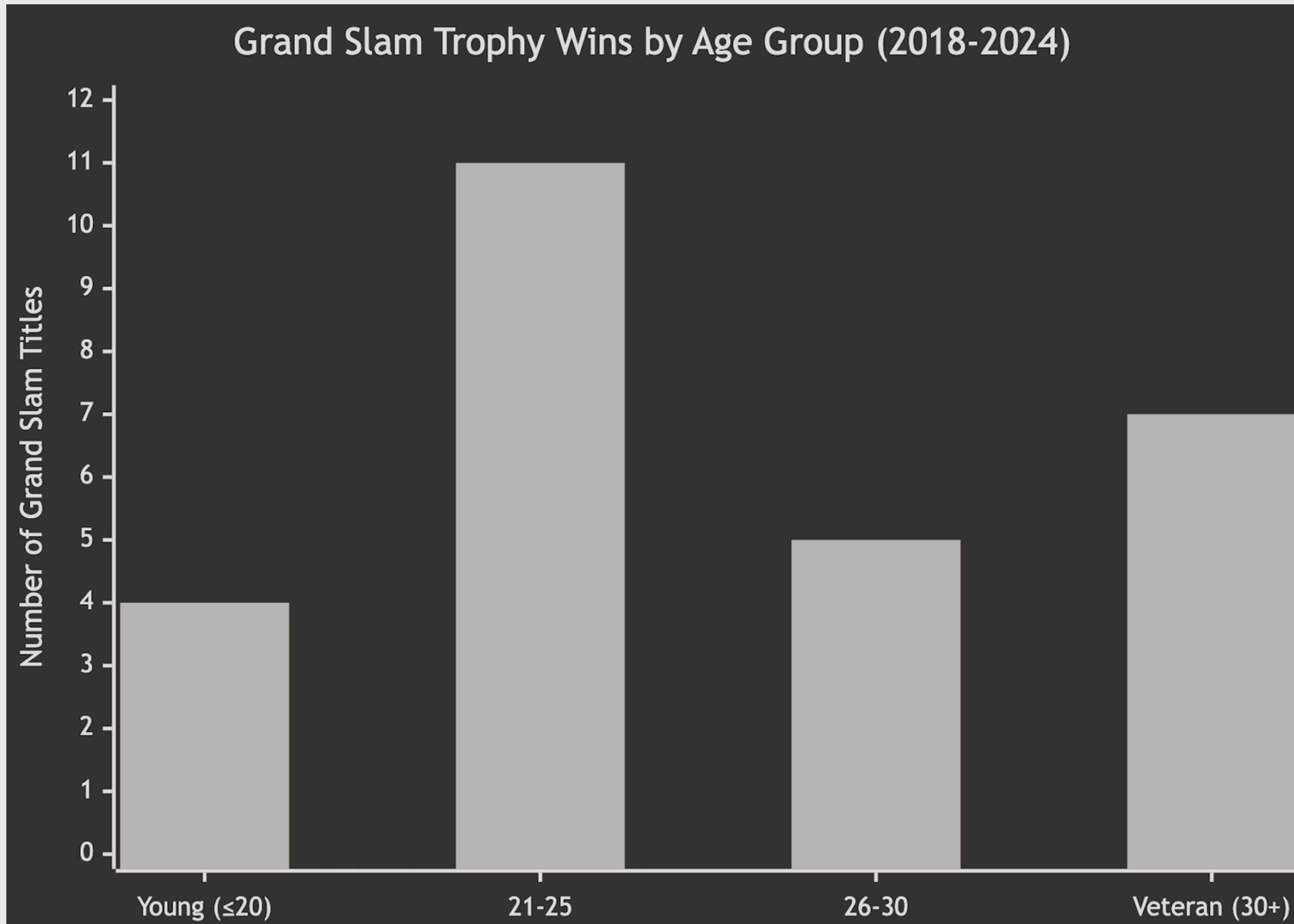
```
FOR PRESSURE SITUATION PERFORMANCE BY AGE GROUP:
WITH match_players AS (
  -- Using normalized tables
  SELECT
    pm.tourney_id,
    pm.match_num,
    pm.player_id,
    pm.role,
    TIMESTAMPDIFF(YEAR, p.dob, t.start_date) AS age,
    m.round
  FROM player_match pm
  JOIN players p ON pm.player_id = p.player_id
  JOIN matches m ON pm.tourney_id = m.tourney_id AND pm.match_num = m.match_num
  JOIN tournaments t ON pm.tourney_id = t.tourney_id
  WHERE m.round IN ('QF', 'SF', 'F', 'RR') -- Quarterfinals and beyond
),
age_groups AS (
  SELECT
    *,
    CASE
      WHEN age <= 20 THEN 'Young (≤20)'
      WHEN age BETWEEN 21 AND 25 THEN '21-25'
      WHEN age BETWEEN 26 AND 30 THEN '26-30'
      ELSE 'Veteran (30+)'
    END AS age_group
  FROM match_players
)
SELECT
  age_group,
  COUNT(*) AS total_matches,
  SUM(CASE WHEN role = 'W' THEN 1 ELSE 0 END) AS wins,
  ROUND(SUM(CASE WHEN role = 'W' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS win_percentage,
  -- Round-specific breakdown
  ROUND(SUM(CASE WHEN role = 'W' AND round = 'QF' THEN 1 ELSE 0 END) * 100.0 /
    NULLIF(SUM(CASE WHEN round = 'QF' THEN 1 ELSE 0 END), 0), 2) AS QF_win_pct,
  ROUND(SUM(CASE WHEN role = 'W' AND round = 'SF' THEN 1 ELSE 0 END) * 100.0 /
    NULLIF(SUM(CASE WHEN round = 'SF' THEN 1 ELSE 0 END), 0), 2) AS SF_win_pct,
  ROUND(SUM(CASE WHEN role = 'W' AND round = 'F' THEN 1 ELSE 0 END) * 100.0 /
    NULLIF(SUM(CASE WHEN round = 'F' THEN 1 ELSE 0 END), 0), 2) AS F_win_pct
  FROM age_groups
  GROUP BY age_group
  ORDER BY
    CASE age_group
      WHEN 'Young (≤20)' THEN 1
      WHEN '21-25' THEN 2
      WHEN '26-30' THEN 3
      ELSE 4
    END;
END;
```

SQL CODE

FOR GRAND SLAM TROPHY WINS BY AGE GROUP:

```
SELECT
CASE
  WHEN winner_age <= 20 THEN 'Young (≤20)'
  WHEN winner_age BETWEEN 21 AND 25 THEN '21-25'
  WHEN winner_age BETWEEN 26 AND 30 THEN '26-30'
  ELSE 'Veteran (30+)' END AS age_group,
COUNT(*) AS grand_slam_trophies,
COUNT(DISTINCT winner_id) AS unique_champions
FROM stg_matches_raw
WHERE round = 'F'
AND (tourney_name LIKE '%Australian Open%' OR
tourney_name LIKE '%French Open%' OR
tourney_name LIKE '%Roland Garros%' OR
tourney_name LIKE '%Wimbledon%' OR
tourney_name LIKE '%US Open%')
AND FLOOR(tourney_date / 10000) BETWEEN 2018 AND 2024
GROUP BY
CASE
  WHEN winner_age <= 20 THEN 'Young (≤20)'
  WHEN winner_age BETWEEN 21 AND 25 THEN '21-25'
  WHEN winner_age BETWEEN 26 AND 30 THEN '26-30'
  ELSE 'Veteran (30+)'
END
ORDER BY grand_slam_trophies DESC;
```



ADVANTAGES OF OLDER PLAYERS

- **Higher aces per match**
- **More service games held**
- **More break points saved**

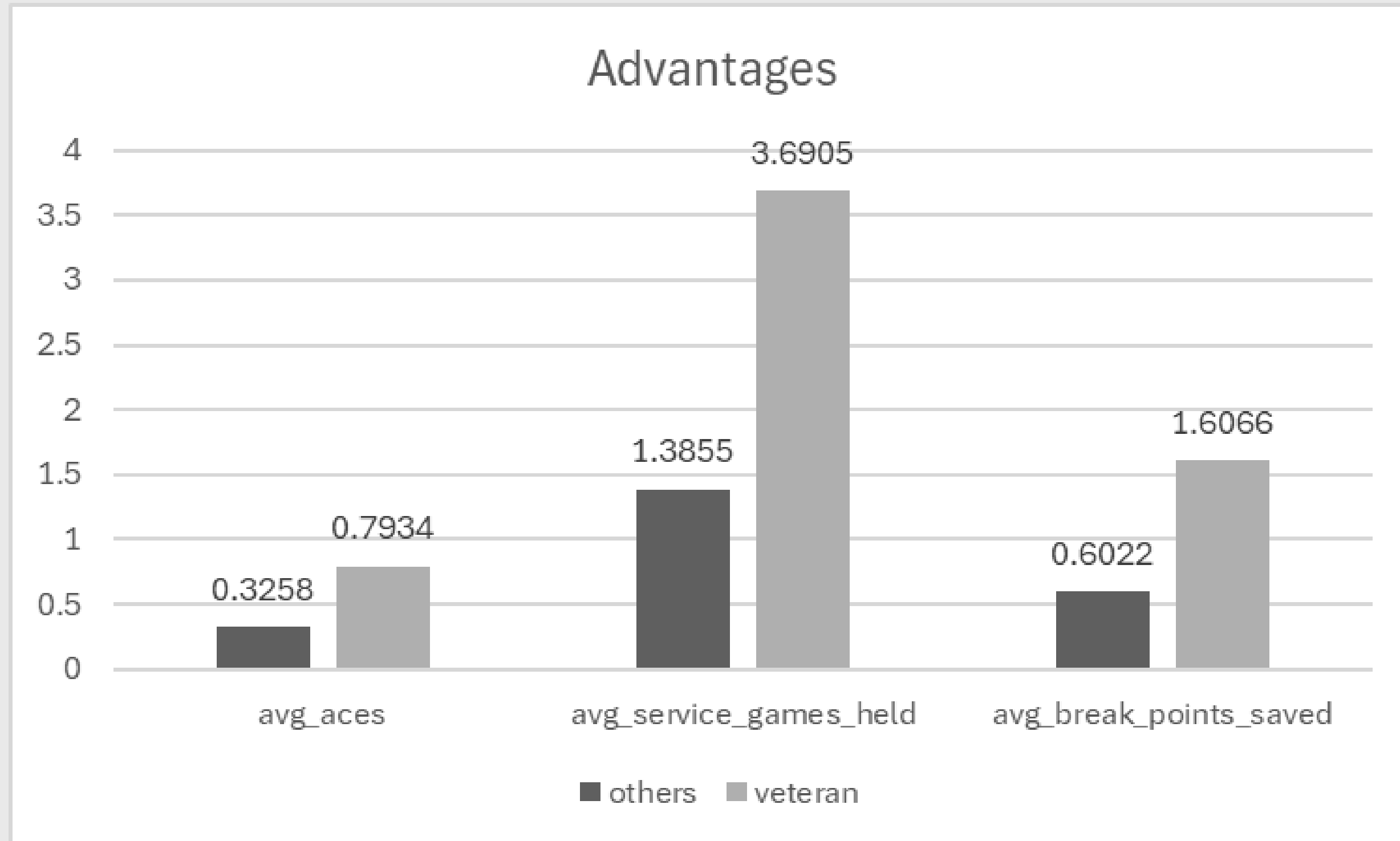
SQL CODE & OUTPUT

FOR STATISTICS FAVORING OLDER PLAYERS:

```
SELECT
  age_group,
  AVG(ace) AS avg_aces,
  AVG(sv_gms) AS avg_service_games_held,
  AVG(bp_saved) AS avg_break_points_saved
FROM (
  SELECT
    ms.ace,
    ms.sv_gms,
    ms.bp_saved,
    CASE
      WHEN (LEFT(sm.tourney_date, 4) - LEFT(p.dob, 4)) >= 33 THEN 'veteran'
      ELSE 'others'
    END AS age_group
  FROM players p
  JOIN match_stats ms ON ms.player_id = p.player_id
  JOIN stg_matches_raw sm ON ms.match_num = sm.match_num AND ms.tourney_id = sm.tourney_id
) sub
GROUP BY age_group;
```


OUTPUT

age_group	avg_aces	avg_service_games_held	avg_break_points_saved
others	0.3258	1.3855	0.6022
veteran	0.7934	3.6905	1.6066



DISADVANTAGES OF OLDER PLAYERS

- **Higher double faults**
- **Longer match duration**
- **First serve count**

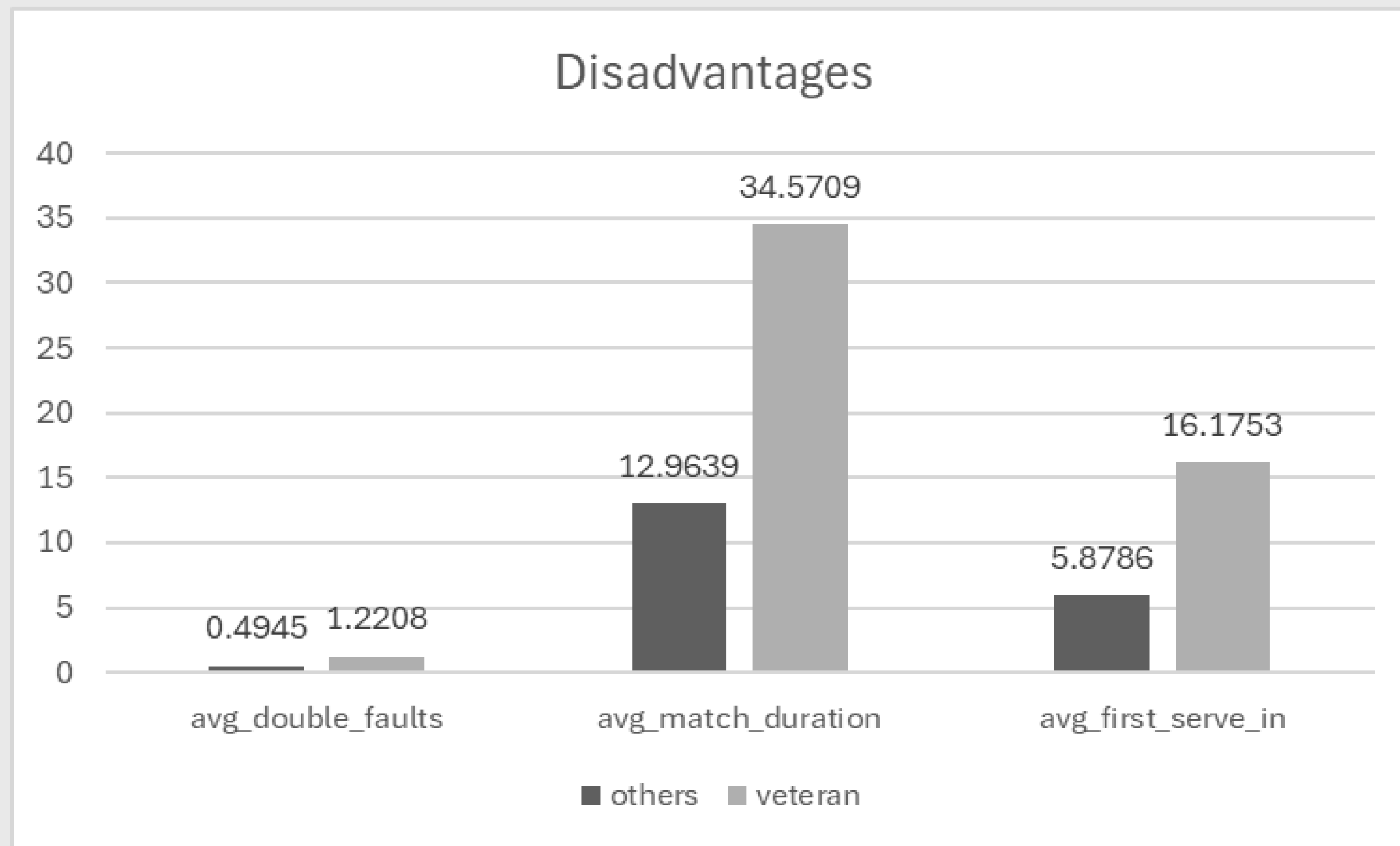
SQL CODE & OUTPUT

FOR STATISTICS NOT FAVORING OLDER PLAYERS:

```
SELECT
  age_group,
  AVG(df) AS avg_double_faults,
  AVG(first_in) AS avg_first_serve_in,
  AVG(minutes) AS avg_match_duration
FROM (
  SELECT
    ms.df,
    ms.first_in,
    sm.minutes,
    CASE
      WHEN (LEFT(sm.tourney_date, 4) - LEFT(p.dob, 4)) >= 33 THEN 'veteran'
      ELSE 'others'
    END AS age_group
  FROM players p
  JOIN match_stats ms ON ms.player_id = p.player_id
  JOIN stg_matches_raw sm ON ms.match_num = sm.match_num AND ms.tourney_id = sm.tourney_id
) sub
GROUP BY age_group;
```

OUTPUT

age_group	avg_double_faults	avg_match_duration(mins)	avg_first_serve_in
others	0.4945	12.9639	5.8786
veteran	1.2208	34.5709	16.1753



MOST INTERESTING AND HELPFUL TOOLS

- We found the attributes the Veterans were the most proficient in the most interesting
- The coding languages used were also very interesting and incredibly helpful

CONCLUSION AND REFLECTION

- There are many fascinating observations and conclusions that can be made regarding tennis due to analytics generated
- While the overall prime age range is between 24-26 years old, there are certain advantages that both younger and older players possess
- More familiarity with the data analysis and visualization tools used would have been beneficial
- Comparing current to historical data regarding a player's prime would have served as valuable contextual data
- Meeting as a group in person more often may have improved the cohesiveness of the team

THANK YOU

